

基本操作

對齊快速鍵：可單行或整段選取來進行操作

Tab 鍵：往右對齊。**Shift + Tab 鍵**：往左對齊。

變數：主要有**整數**、**字串**、**串列**、**字典**、**集合**。

不可變物件：創建後，物件的值不可改變，如果試圖修改值，會生成一個新的物件。

常見的不可變物件：**整數(int)**、**字串(str)**、**元組(tuple)**

```
1 '''把x看成是便利貼，10存到記憶體後，把x指到這個記憶體
2    10+5=15，產生的15存到新的記憶體位址，把x指到新的記憶體
3    10、15存到記憶體後(程式結束前)，這些值、位址都不會改變。
4    '''
5    x = 10
6    print(id(x))    # x的記憶體位址：2292307159632
7    x += 5          # 創建新物件並重新賦值，x移到15的記憶體位址
8    print(id(x))    # x ID 改變：2292307159792
```

可變物件：創建後，物件的值可以被修改，而不會改變物件的 id(記憶體位址)

常見的可變物件：**串列(list)**、**字典(dict)**、**集合(set)**

id()函式：查詢變數的記憶體位址。

```
1 '''lst建立後，存在記憶體中的其實為3個指向別的記憶體位址的值
2    lst的ID不會變，但內部的指向值會變。
3    但只討論lst的ID值。所以內部任何的新增、刪除、修改，都不會變。
4    '''
5    lst = [1, 2, 3]
6    print(id(lst))    #記憶體位址：1939148980160
7    print(id(lst[1])) #內部第2索引指的位址：1939148532048
8    lst[1]=10         #修改內部第2索引的值
9    print(id(lst[1])) #新的第2索引指的位址：1939148532304
10   lst.append(4)     # 修改列表內容
11   print(id(lst))    # 物件 ID : 1939148980160
```

學科常考題：函式傳參數時。

不可變物件不可修改，原始物件不受影響。但可使用 **global** 宣告為全域變數。

可變物件可以直接修改，會影響到原始物件。不需使用 **global** 即為全域變數。

補充說明：若可變物件使用 **global**，則表示為重新給一個新物件。

不可變物件，函式遞後的值：

<pre>1 a = 10 2 def fn_1(): 3 global a 4 a = 1 5 def fn_2(): 6 a = 1 7 fn_1() 8 print(a) 9 fn_2() 10 print(a)</pre>	<pre>1 a = 10 2 def fn_1(): 3 a = 1 4 def fn_2(): 5 global a 6 a = 1 7 fn_1() 8 print(a) 9 fn_2() 10 print(a)</pre>
---	---

可變物件，函式遞後的值：

<pre>1 a=[1,2,3] 2 def fn_1(): 3 a.append(4) 4 def fn_2(): 5 a=[1,2] 6 fn_1() 7 print(a) 8 fn_2() 9 print(a)</pre>	<pre>1 a=[1,2,3] 2 def fn_1(): 3 a.append(4) 4 def fn_2(): 5 global a 6 a=[1,2] 7 fn_1() 8 print(a) 9 fn_2() 10 print(a)</pre>
--	--

運算子

算術運算子(依優先順序):

括號	指數	乘、除、整數除、餘數	加、減
()	**	* / // %	+ -

賦值運算子(讓程式更潔簡): +=、-=、*=、/=、//=、%=、**=

比較運算子：產生的結果為 True 或 False : ==、!=、>、>=、<、<=

邏輯運算子：產生的結果為 True 或 False: and、or、not

位元運算子： 先將整數轉化成二進位再進行運算

&(AND)、|(OR)、~(NOT, 補數)、^(XOR)、

>>(向右移一位, 右邊減 1 個位元, 左邊加 1 個 0), <<(向左移一位, 右邊加 1 個 0)

4(二進位 0100)、5(二進位 0101)

print (4 & 5) # 4 (回傳 0100)	print (~4) # -5 (回傳 1011)
print(4 5) #5 (回傳 0101)	print(4 >>2) #1 (回傳 0001)
print (4 ^ 5) # 1 (回傳 0001)	print (5<<2) # 20 (回傳 10100)

成員運算子(in、not in): 檢查某值是否為集合的一部份。結果為 True 或 False

```
1 a=[1,2,3]
2 print(3 in a)      #True
3 print(4 not in a)  #True
```

身分運算子(is、is not): 用於比較兩個物件的記憶體位置。結果為 True 或 False

```
1 x=10
2 y=10
3 a=[1,2,3]
4 b=[1,2,3]
5 print(x is y)      #True
6 print(a is b )     #False
```

函數整理-常用

input(): 輸入，預設為字串，可用 int()、eval()來轉換。

print(): 輸出，會自動換行。資料輸出不換行格式。print('test',end='') 放空字串即可。

函數整理-數值

int(): 將資料型態強制轉為整數。如果無法轉換會出現錯誤訊息(程式會中斷)。

int(x 字串,n 進位值): 將 x 字串,n 進制的值=>轉為 10 進制。eval(): 將字串，去掉前後的單或雙引號，常用於資料輸入時的轉換。float(): 將資料型態強制轉為浮點數。

bin(): 將整數數字轉換為 2 進位。

oct(): 將整數數字轉換為 8 進位。

hex(): 將整數數字轉換為 16 進位。

abs(): 絕對值。

round(f,n): 浮點數 f，取到小數第 n 位。

bool(): 將資料轉成 True 或 False。數值如果是 0 或空資料，視為 False。

函數整理-字串(有些函數也通用於串列、字典、集合)

str(): 強制將數值資料轉為字串資料。chr(x):

傳回 x 字元符號對應的 ASCII 碼值。ord(n):

傳回 ASCII 數值 n 對應的字元符號。lower():

將字串內的字母轉成小寫。upper(): 將字串內的字母轉成大寫。

title(): 將字串的第一個字母轉成大寫，其它是小寫。

strip():

刪除字串頭尾兩邊多餘的空白。rstrip(): 刪除字尾的空白，lstrip(): 刪除開頭的空白 split():

將字串用空格或指定符號來切割，切割後儲存到串列中。

string.split() #用空格切割，string.split(',') #用逗號切割，input().split() #資料輸入即切割

len(): 取得字串長度。

find(): 從左邊開始，回傳第一個找到的字串內指定字元的索引值，無則回傳-1
 index(): 同 find(), 但如沒有找到會出現錯誤訊息。建議用 find() count():
 回傳子字串出現的次數
 startswith(substring): 字串起始文字是否為指定文字。回傳 True or False
 endswith(substring): 字串結束文字是否為指定文字。 replace(old,new):
 將原 old 字串由新 new 字串取代。(只要符合就會取代) isXXX 系列函數:
 判斷後回傳 True or False
 islower(): 判斷字串中的字母是否全部小寫 isupper():
 判斷字串中的字母是否全部大寫 isalpha(): 判斷字串
 只含有字母，且不是空字串 isdigit(): 判斷字串只含
 有數字，且不是空字串 isalnum(): 判斷字串只有字
 母或數字，且不是空字串
 istitle(): 判斷字串是否符合標題格式(每個單字的首字大寫，其餘都是小寫)

格式輸出

f-string 格式: 競賽統一用這種輸出格式。
 以 f 為字首，在大括號{}內放置變數名稱或運算式。

```
1 a=int(input())          # '5'
2 print(a)                # 5
3 print(f'a={a}')         # a=5
4 print(f'a^2={a**2}')    # a^2=25
```

format(): 用來格式化字串的函式或方法，可以將數值、字串或其它資料型轉成物定格式。

- (1)格式化數字，轉成指定的n進位(b:二進位、o:八進位、x:16進位(小寫)、X:16進位(大寫))
- (2)補零與對齊：補足位數(前導補0)或控制對齊
- (3)數字格式化(千位分隔符、浮點數)

f-string與format的結合

```
1 num = 90
2 b_num = f'{num:010b}'
3 print(b_num) #二進位 0001011010
4 print(f'{num:010o}') #八進位 0000000132
5 print(f'{num:010x}') #十六進位 000000005a
6 print(f'{num:010X}') #十六進位 000000005A
7 print(f'{123456:,d}') #十進位加千分號 123,456
8 print(float(f'{3.1415:.2f}')) #取小數二位數
```

特殊資料型態：串列(list)

串列內的資料型態可以不一樣。索引值從 0 開始。

串列內的每一筆資料稱為元素，用中括號[]表示一個元素。用逗號隔開

```
1 a = [] #建立空字串
2 b = [1,2,'a',[0,0]] #元素的資料型態可以不一樣
3 c = [i for i in range(4)] #串列生成式 [0,1,2,3]
4 d = [0 for j in range(4)] #串列生成式 [0,0,0,0]
5 e = [[i for i in range(4)] for j in range(3)] #二維串列
6 # [[0, 1, 2, 3], [0, 1, 2, 3], [0, 1, 2, 3]]
7 print(e[0]) # [0, 1, 2, 3] 索引值0的值是一個串列
8 print(e[1][2]) # 2 索引值1的串列內的索引值2的值
```

串列切片(用字串，字串也是串列的一種，但字串的值不能利用切片來修改)

切片公式：串列變數名稱[start:edn:step]

x[start:end]: 從索引值 start 到 end-1 索引值內的元素

x[:end]: 從索引值 0 到 end-1 索引值內的元素 x[:]:

取得所有元素。y=x[:]等同於 y=list(x)

x[::-1]: 反向取得所有元素。

x[start:end:step]: 每隔 step 個的元素。

反向索引值：從後面開始的索引值

索引值	0	1	2	3	4
內容值	10	24	8	15	30
反向索引值	-5	-4	-3	-2	-1

串列函數，使用上述資料 `x=[10,24,8,15,30]`。

`max()`: 取得串列的最大值。 `max(x)` # 30

`min()`: 取得串列的最小值。 `min(x)` # 8

`sum()`: 取得串列的總和。 `sum(x)` # 87

`len()`: 取得串列的元素個數。 `len(x)` # 5

`del`: 刪除串列內的元素

`del x[1]` # `x = [10,8,15,30]`

`del x[1:3]` # `x = [10]` `append()`:

在串列最後端，增加一個新的元素

`x.append(50)` # `x = [10,50]`

`pop(n)`: 取出指定的索引的值，並刪除。省略 `n` 表示最後一個。

`y = x.pop(0)` # `y = 10` `x=[50]` `remove()`:

刪除第一個符合指定的元素內容。

`x.remove(50)` # `x=[]`

重設 `x = [10,24,8,15,30]`

`reverse()`: 反向整個串列元素。 `x.reverse()` # `[30, 15, 8, 24, 10]` 同 `x = x[::-1]`

`sort()`: 將串列元素由小到大排序。 `x.sort()` # `[8, 10, 15, 24, 30]` 結果回存 `x`

`sorted()`: 排序後儲存到新串列。 `y = sorted(x,reverse = False)` # `[8, 10, 15, 24, 30]`

`reverse = False` 可省略，`True` 為由大到小

`index()`: 傳回指定元素值第一次出現的索引值。 `a = x.index(15)` # `a=2`

`count()`: 傳回指定元素值出現的次數。 `b = x.count(10)` # `b=1`

特殊函數 `zip()`: 英文意思為拉鍊，在 `list` 串列中，迭代多個 `list`，每次分別取出對應的元素組成一組回傳。

```

1 s1=["a","b","c"]
2 s2=[1,2,3,4]
3 for i,j in zip(s1,s2):
4     print(i,j) #a 1 b 2 c 3

```

特殊資料型態：字典(dict)，非序列資料結構，沒有索引值。

元素用「鍵(key): 值(value)」方式配對儲存。

一、鍵：值放在大括號 { } 內。利用鍵來取得值。

```

1 a = { }
2 b = {'a':'apple','b':'bear'}
3 c = {i : i*i for i in range(1,3)} # {1: 1, 2: 4}
4 print(b['b']) # bear
5 print(c[2]) # 4

```


二、字典的新增(key 不存在就增加)、更改(key 已存在就修改)、刪除、合併

`del x[鍵]` #刪除指定的鍵，若不存會出現錯誤訊息，可先用 `in` 檢查。

`pop`: 刪除指定的鍵，同時傳回對應的值。

`update`: `x.update(y)` #y 合併至 x，若重覆 key，則 value 被覆蓋。

```
1 a = {1:'a',2:'b'}
2 a[5] = 'c' # {1: 'a', 2: 'b', 5: 'c'}
3 a[1] = 'd' # {1: 'd', 2: 'b', 5: 'c'}
4 del a[2] # {1: 'd', 5: 'c'}
5 b = {5:'t',4:'p'}
6 a.update(b) # {1: 'd', 5: 't', 4: 'p'}
7 x = a.pop(5) # x=t a = {1: 'd', 4: 'p'}
```

三、取得字典內的資料 `items()`方法，回傳 1 個元組，用二個變數分別取出。

```
1 x={"a":1,"b":2,"c":3}
2 for key,value in x.items(): #鍵、值皆讀取
3     print(key,value) #a 1 b 2 c 3
4 for key in x.keys(): #只讀取鍵
5     print(key) #a b c
6 for value in x.values(): #只讀取值
7     print(value) # 1 2 3
```

四、字典排序。因為字典沒有索引值，所以利用 `sorted()`來排序。

```
1 old={"a":2,"c":1,"d":2,"b":3}
2 #old.get表示取出value值。
3 list_k=sorted(old) # ['a', 'b', 'c', 'd'] , 用key來排序
4 list_v=sorted(old,key = old.get) # ['c', 'a', 'd', 'b'] , 用value來排序
```

特殊資料型態：集合(set)：沒有索引值，且每個元素是唯一的。

一、建立集合，同字典一樣使用 { } 大括號，但只有鍵，沒有值。

二、集合的函式：`len()`、`max()`、`min()`、`pop()`、`remove()`使用方法同 list。

唯一不同的是新增元素函式：`add()`

```
1 x = {2,4,3,4,2,1}
2 print(x) # {1,2,3,4}
3 x.add(6) # {1,2,3,4,6}
```

三、`set()`、`list()`資料型態可以互轉。一般是為了可以使用 list 的函式。

```
1 a = 'book'
2 b = set(a) # b = {'b', 'o', 'k'}
3 c = list(b) # c = ['k', 'b', 'o']
4 c.sort() # c = ['b', 'k', 'o']
```

小結：串列，字典，集合的資料型態類似，但因為串列有索引值，而字典、集合沒有，所以使用時查詢、插入、刪除時，字典、集合的會比串列好。

For 迴圈 range(start,end,step) start <=變數 < end

1、物件內(字串、串列)的資料依序取出每一個元素。

```
1 s = 'abc'
2 t = [1,2,3]
3 for i in s:
4     print(i) # a b c
5 for j in t:
6     print(j) # 1 2 3
```

2、利用切片，指定取出的範圍

```
1 s="abcde"
2 t=[1,2,3,4,5]
3 for i in s[2:4]:
4     print(i) # c d
5 for j in t[2:4]:
6     print(j) # 3 4
```

3、強制離開 break, continue: 跳過該次的剩餘部份，直接進入下一次迴圈。

```
1 for i in range(1, 8):
2     if i % 3 == 0:
3         continue
4     print(i) # 1 2 4 5 7
5 for i in range(1, 8):
6     if i == 5:
7         break
8     print(i) #1 2 3 4
```

4、for else 迴圈：在迴圈正常執行完畢且未被 break 終止時，執行額外的操作。

```
1 for item in iterable:
2     if condition:
3         break # 中斷迴圈時，不執行 else 的內容
4     # 其他程式碼
5 else:
6     # 當迴圈正常執行完成（沒有遇到 break）時執行

1 # 判斷的數字是否為質數 num >=2
2 num = int(input("請輸入一個大於 1 的整數："))
3 for i in range(2, int(num ** 0.5) + 1):
4     if num % i == 0:
5         print(f"{num} 不是質數，因為它可以被 {i} 整除")
6         break
7 else:
8     print(f"{num} 是質數")
```

5、enumerate()函數，同時取出索引值和內容值，較 range 方便、快速。

```
1 b=[1,2,3]
2 for x,y in enumerate(b):
3     print(x,y) #x(索引值) y(內容值) 0 1 1 2 2 3
```


while 迴圈：條件控制迴圈，觀念同 for。只是不知道會執行幾次。

1、while 條件公式

while 變數 in 物件 #變數值存在物件中為 True

while 物件 #物件是空的才結束

2、while - continue: 不往下執行，回到 while 第一行動新執行

3、while – break: 迴圈強制結束離開

4、while – else: 迴圈正常結事時會執行 else 的程式碼，但 break 跳離迴圈時不會執行。

正式選手應學技巧

一、內建函式庫，使用時需要匯入

(1) import math: 數學函式庫

```
print(math.ceil(9.5)) #10 向上取整數
print(math.floor(9.5)) #9 向下取整數
print(math.sqrt(9)) #平方根 = **0.5
print(math.dist((1,2),(4,6))) #計算二點之間的距離
print(math.pow(2,3)) #2的3次方 = 2**3
print(math.exp(2)) #自然對數函數
print(math.log(10,2)) #用2為底的對數
print(math.pi) #圓週率
```

(2) import itertools: 排列組合函式庫。

list 表示一個原始串列的資料，r 表示要選取幾個值

itertools.permutations(list,r): 返回長度為 r 的所有排列組合

itertools.combinations(list,r): 返回長度為 r 的所有組合

```
import itertools
a=[1,2,3]
p_a=itertools.permutations(a,3)
print(*p_a) #6種排列 123 132 213 231 312 321
c_a=itertools.combinations(a,2)
print(*c_a) #3種組合 (1, 2) (1, 3) (2, 3)
```

二、多重指定，已知字串或串列的長度時可用多重指定

```
1 s = '123'
2 a,b,c = s
3 print(a,b,c) # 1 2 3 (字串)
4 k = [1,2,3]
5 x,y,z = k
6 print(x,y,z) # 1 2 3 (整數)
```

三、將字串直接轉成串列，將串列直接轉成字串

```
1 s = '12345'
2 a = list(s)          #用list()將字串轉換為型態，
3 print(a)             # ['1', '2', '3', '4', '5']
4 x = [1,2,3,4,5]
5 str_x = map(str,x)   #要先把元素的資料型態轉成字串
6 y = ''.join(str_x)  #利用join()字串函數，將串列轉成字串
7 print(y)             # 12345
```

四、lambda 函式用法

1、lambda 函式只有一行，包含三個部份。lambda 參數清單 運算式

```
1 #表示有二個參數，這二個參數執行相乘的運算
2 multiply=lambda x,y:x*y
3 print(multiply(4,2))      #8
4 print((lambda x,y:x*y)(4,2)) #8
```

2、用 filter()與迭代物件配合使用。每個元素判斷過濾後，存到 result 串列

```
1 numbers=[50,2,12,30,27,4]
2 result=list(filter(lambda x:x>10,numbers))
3 print(result) # [50, 12, 30, 27]
```

3、用 map()與迭代物件配合使用。每個元素讀取進行運算式

```
1 numbers=[50,2,12,30,27,4]
2 result=list(map(lambda x:x*2,numbers))
3 print(result) #[100, 4, 24, 60, 54, 8]
```

4、lambda 存在一個函數內部

```
1 def func(b):
2     return lambda x:2*x+b
3 linear = func(5)
4 print(linear(10)) # 25
```

5、字典排序使用 lambda

```
1 old={"a":2,"c":1,"d":2,"b":3}
2 # old.items(),會返回字典old的所有鍵值組資料(a,2) (c,1) (d,2) (b,2)
3 # 用產生串列的元組中第1個索引當排序依據
4 new1=sorted(old.items(),key=lambda x:x[0]) #用鍵重新排序字典
5 # 用產生串列的元組中第2個索引當排序依據
6 new2=sorted(old.items(),key=lambda x:x[1]) #用值重新排序字典
7 # 排序依據，先用值x[1]，如相等再用鍵x[0]
8 new3=sorted(old.items(),key=lambda x:(x[1],x[0]))
9 print(new1) #[('a', 2), ('b', 3), ('c', 1), ('d', 2)]
10 print(new2) #[('c', 1), ('a', 2), ('d', 2), ('b', 3)]
11 print(new3) #[('c', 1), ('a', 2), ('d', 2), ('b', 3)]
```

五、數學集合的操作：交集、聯集、差集、對稱差集。注意使用的特殊符號

```
1 a={1,2,3,4}
2 b={4,5,6,7}
3 print(a&b)      #交集 {4}
4 print(a|b)      #聯集 {1, 2, 3, 4, 5, 6, 7}
5 print(a-b,b-a)  #差集 {1, 2, 3} {5, 6, 7}
6 print(a^b)      #對稱差集 {1, 2, 3, 5, 6, 7}
7 print(a==b)     # False
8 print(a!=b)     # True
```

6、二元樹

Class: 建一個二元樹 類別名稱第一個字母建議用大寫(只有二元樹會使用類別)

(1)、先定義二元樹的類別

```
1 class TreeNode:    #樹節點資料結構
2     #初始化設定(固定寫法)，self表示自己，data表示傳入一個數值
3     def __init__(self,data):
4         self.value=data    #儲存傳入的值
5         self.left=None    #左子樹的節點位址，預設是空的
6         self.right=None   #右子樹的節點位址，預設是空的
```

(2)、建立二元樹，遞迴程式，離開的條件是左、右節點是空的

```
7 def insert(root,value):    #二個參數，已存在的樹節點，新加入的值
8     if root is None:       #還沒有加入樹的新節點
9         return TreeNode(value)    #新節點建立
10    '''新加入的值比目前的樹節點小，往左邊移動
11    傳左子樹節點位址，要加入的值'''
12    if value < root.value:
13        root.left=insert(root.left,value)
14    else: #觀念同上，只是往右邊
15        root.right=insert(root.right,value)
16    return root    #完成後，要回傳
```

(3)、主程式

```
root = TreeNode(7)    #建立根節點 7
nodes = [4,1,5,12,8,9,15]    #要加入的節點
tree_way = []    #收集走過的節點，串列才可以全域
for node in nodes:    #將節點依序加入到二元樹
    insert(root,node)
tree_travel(root)    #進行拜訪
print(*tree_way)    #輸出結果，串列轉字串可用join
```

(4)、前、中、後序的各種輸出。遞迴程式要了解為什麼這樣執行

```
17 def tree_travel(tree):
18     if tree is not None: #還有子節點
19         tree_way.append(tree.value) #前序
20         tree_travel(tree.left) #往左節點移動
21         #tree_way.append(tree.value) #先左移，是中序
22         tree_travel(tree.right) #往右節點移動
23         #tree_way.append(tree.value) #左、右移後，是後序
```